

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE
COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A **APPLICATION BASED QUESTIONS**

Code of Conduct:

I syed arshad /192525255 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Arshad Khadri

Q1. Smart City Surveillance — CPU/Memory/I/O Interaction & Bus Structures

Core idea to build the answer around

Video surveillance generates continuous high-volume data. Lag during peak hours happens when the path connecting CPU, memory, and I/O devices becomes a bottleneck — either because a single shared bus is being fought over by too many devices, or because the instruction cycle spends too much time waiting on slow memory/I/O accesses.

How to explain the CPU–Memory–I/O interaction

- Camera feeds arrive as I/O input fi must be moved into memory (buffers) fi CPU fetches/processes frames for threat detection fi output/alerts go back through I/O.
- Each of these transfers competes for the same bus if there is only one shared pathway (single-bus architecture).
- During peak hours, more cameras = more DMA/I/O requests + more memory reads/writes + more CPU fetch cycles happening simultaneously, so contention increases and effective throughput drops.
- This shows up as increased memory access latency and queuing delay, which directly slows the fetch–decode–execute cycle.

Single-bus vs Multi-bus structures (this is the comparison the question wants)

Single-Bus Architecture:

Structure: One shared address/data/control bus used by CPU, memory, and I/O

Contention: High — only one transfer can happen at a time

Cost: Cheaper and simpler to design

Suitability: Poor for real-time multi-camera load

Multi-Bus Architecture:

Structure: Separate buses (e.g., dedicated memory bus, I/O bus, cache bus)

Contention: Low — parallel transfers are possible

Cost: More expensive, more complex control logic

Suitability: Well suited for high-throughput, concurrent I/O like surveillance systems

Improvements to propose

- Move to a multi-bus or hierarchical bus structure so camera I/O traffic doesn't block CPU–memory traffic.
- Use DMA (Direct Memory Access) so video data goes I/O to memory directly without CPU supervising every byte, freeing the CPU for actual threat-detection computation.
- Reduce fetch–decode–execute overhead: pipelining, instruction/data cache to cut memory-access wait states, and interrupt-driven I/O instead of polling.
- Buffering/queuing strategy so bursts of camera data don't stall the bus instantaneously.

Supporting code — simulating bus contention (add this as a small demonstration)

```
#include <stdio.h>
```

```

int main() {
    int cameras = 6;
    int bus_cycle_time = 2;

    int single_bus_delay = cameras * bus_cycle_time;
    printf("Single-bus total delay = %d cycles\n",
single_bus_delay);

    int io_bus_delay = (cameras / 2) * bus_cycle_time;
    int mem_bus_delay = (cameras / 2) * bus_cycle_time;
    int multi_bus_delay = (io_bus_delay > mem_bus_delay) ?
io_bus_delay : mem_bus_delay;
    printf("Multi-bus total delay = %d cycles\n", multi_bus_delay);

    return 0;
}

```

Q2. Stock Trading System — CPI, Fetch–Decode–Execute, Reducing CPI

Core idea

Millions of instructions/sec but still delayed transactions points to a high CPI (Cycles Per Instruction) problem, not a raw clock-speed problem. Frequent memory access and branch instructions are the two classic causes of CPI inflation.

Key formula to quote

Execution Time = Instruction Count × CPI × Clock Cycle Time.

Since instruction count is already high (millions/sec), the lever available here is reducing CPI (and/or clock cycle time), not reducing instructions.

How memory access and branches raise CPI

- **Fetch stage:** every instruction fetch is itself a memory access — if data also needs frequent memory access (cache miss), the CPU stalls waiting for data from slower main memory.
- **Decode stage:** normally fast, not usually the bottleneck.
- **Execute stage:** branch instructions (if/else, loops) disrupt the normal sequential flow — the CPU may fetch the wrong next instruction and must flush and refetch, wasting cycles (this is a branch penalty).
- **Together:** memory stalls + branch mispredictions push average CPI well above the ideal value of

TECHNIQUE	HOW IT HELPS
Pipelining	Overlaps fetch/decode/execute of multiple instructions instead of doing them serially.
Cache memory	Cuts the cost of frequent memory access by keeping hot data close to the CPU.
Branch prediction	Predicts the likely branch outcome so the pipeline doesn't stall/flush as often
Superscalar execution	Issues more than one instruction per cycle where dependencies allow.

Out of order execution	Reorders instructions to keep execution units busy while waiting on memory.
------------------------	---

Supporting code — calculating CPI and execution time

```
#include <stdio.h>
```

```
int main() { long instruction_count = 5000000; float clock_cycle_time
= 0.5e-9;
```

```
float    cpi_alu      =    1.0,    freq_alu      =    0.5;
float    cpi_mem      =    4.0,    freq_mem      =    0.3;
float    cpi_branch   =    3.0,    freq_branch   =    0.2;
```

```
float effective_cpi = (cpi_alu * freq_alu) + (cpi_mem * freq_mem) +
(cpi_branch
          *
          freq_branch);
float  exec_time   =  instruction_count  *  effective_cpi  *
clock_cycle_time;
```

```
printf("Effective    CPI    =    %.2f\n",    effective_cpi);
printf("Execution    Time    =    %.4f    seconds\n",    exec_time);
return 0;}
```

Q3. 8085 Temperature Control — Addressing Modes & Reliability

Core idea

Incorrect temperature readings in an 8085-based embedded system usually trace back to the program accessing the wrong memory location, wrong register, or a stale/incorrect operand value — which is exactly what addressing modes control.

8085 Addressing modes to define and use

mode	description	Risk if misused
------	-------------	-----------------

Immediate	Operend given directly in instruction	Coded value doesn't reflect data
register	Operend in CPU register	Using stale register value instead of refreshed reading
direct	Operend address given directly	Wrong fixed address reads unrelated memory location
indirect	Address held in register pair	Reads wrong buffer location

Connecting addressing-mode misuse to the actual fault

- If the program uses direct addressing with a fixed/incorrect memory address for the sensor input port, it will keep reading an unrelated memory cell instead of the live sensor value.
- If indirect addressing via HL is used but HL is not incremented/reset correctly between readings, the program reads the previous cycle's stale value.
- Immediate addressing used where a live value was needed (e.g., accidentally hardcoding a threshold) freezes the logic instead of reacting to actual sensor input.

Suggested fix using the instruction set

- Use indirect addressing (MOV A,M with HL pointing at the sensor data buffer) so each cycle reads the correct, current memory location.
- Use IN/OUT instructions correctly matched to the actual port number wired to the sensor.
- Add explicit pointer management (INX H / DCX H) so the buffer pointer is verified before each read.
- Use compare instructions (CPI) and conditional jumps (JZ/JNZ/JC) to validate sensor readings are within an expected range before

triggering the actuator, catching corrupted reads before they cause a wrong action.

Supporting code — 8085 assembly for correct sensor reading

; ---- FAULTY: direct addressing to a fixed address ----

LDA 2050H

; ---- CORRECTED: indirect addressing + range check ----

LXI H, 3000H

MOV A, M

CPI 64H

JNC ALERT

CPI 00H

JZ ALERT

STA 4000H

INX H

JMP CONTINUE

ALERT: MVI A, 0FFH

STA 4001H

CONTINUE:

HLT

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					

Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand